

Lesson 1: JavaScript Fundamentals (Variables, Functions, Loops)

Mission Brief

Welcome aboard the Lumexa Explorer, cadet. Every spaceship needs a working control panel, and every control panel runs on code that remembers things, follows instructions, and repeats tasks without getting tired. Today you become the ship's first JavaScript engineer — learning to store data in variables, package logic into functions, and automate repetition with loops. Without these three tools, nothing else in this course is possible, so strap in.

Learning Objectives

By the end of this lesson, you will be able to:

- Declare variables using `let`, `const`, and explain why `var` is avoided in modern code
- Identify and use JavaScript's primitive data types (string, number, boolean, undefined, null)
- Write and call functions, including arrow functions, with parameters and return values
- Write `for` and `while` loops to repeat actions a specific or conditional number of times
- Use `console.log()` to inspect values while writing and debugging code

What You'll Build

A "Mission Control Console" — a small JavaScript file (run in the browser console and as a `.js` file linked to an HTML page) that stores a spaceship's name, fuel level, and crew list as variables, uses a function to calculate remaining flight time, and uses a loop to print a pre-launch checklist to the console.

Prerequisites

- A modern web browser (Chrome, Firefox, or Edge) with Developer Tools available
- A text editor (VS Code recommended)
- Comfort creating and opening a basic `.html` file (from earlier courses in the Lumexa path)

Key Concepts

- Variable declaration (`let`, `const`)
- Data types: string, number, boolean, undefined, null, array, object
- Operators: arithmetic, comparison, logical
- Functions: declarations, expressions, arrow functions, parameters, return values

- Scope (block scope vs function scope)
- `for` loops and `while` loops
- The browser Console

Concept Explanation

JavaScript is the programming language that makes web pages *do* things — respond, calculate, update, and react. Unlike HTML (structure) and CSS (appearance), JavaScript is a real programming language with variables, logic, and control flow, and it runs directly inside the browser.

A **variable** is a named container for a value. Modern JavaScript gives us two main ways to declare one: `let`, for a value that may change later, and `const`, for a value that should never be reassigned after it's set. You will almost never see `var` in modern code — `var` has confusing scoping rules (it "leaks" outside of blocks like `if` statements and loops), while `let` and `const` are **block-scoped**, meaning they only exist inside the `{ }` curly braces where they were declared. As a rule of thumb: default to `const`, and only use `let` when you know the value needs to change.

JavaScript variables can hold several **data types**. The primitives are `string` (text, wrapped in quotes), `number` (both integers and decimals — JavaScript doesn't separate them), `boolean` (`true` / `false`), `undefined` (a variable that has been declared but not given a value), and `null` (an intentional "no value"). Beyond primitives, JavaScript has `object` (a collection of key-value pairs) and `array` (an ordered list) — you'll use both constantly starting in this lesson.

A **function** is a reusable block of code that performs a task. You can define one with the `function` keyword (`function greet() { }`), or with the more modern **arrow function** syntax (`const greet = () => { }`), which is shorter and behaves slightly differently with the `this` keyword (a topic for a later, more advanced course). Functions can accept **parameters** (inputs) and can `return` a value back to whoever called them. Writing logic inside functions, instead of copy-pasting it everywhere, is one of the most important habits a programmer builds.

Loops let a computer repeat an action without you writing the same line over and over. A `for` loop is ideal when you know how many times you want to repeat something — it has three parts: an initializer (`let i = 0`), a condition (`i < 5`), and an increment (`i++`). A `while` loop repeats as long as a condition stays true, and is useful when you don't know the exact number of repetitions in advance (for example, "keep asking until the user gives a valid answer").

Finally, the browser's **Console** (part of Developer Tools) is your most important tool as a JavaScript engineer. `console.log()` prints a value so you can inspect it — you will use it in nearly every lesson from here on to understand what your code is actually doing.

Guided Coding

Create a file called `mission-console.js` and an `index.html` that loads it, then follow along:

```

// mission-console.js
// Lumexa Mission Control Console – Lesson 1

// --- VARIABLES ---
// const: values that should never be reassigned
const shipName = "Lumexa Explorer";
const crew = ["Ada", "Grace", "Katherine", "Mae"]; // an array of strings

// let: values that will change over the course of the mission
let fuelLevel = 92; // percent
let isLaunchReady = false;

// --- OPERATORS ---
fuelLevel = fuelLevel - 4; // arithmetic: burned some fuel during pre-checks
isLaunchReady = fuelLevel > 80; // comparison operator produces a boolean

console.log("Ship:", shipName);
console.log("Fuel level:", fuelLevel);
console.log("Ready for launch?", isLaunchReady);

// --- FUNCTIONS ---
// A function declaration that calculates flight time from fuel and burn rate.
function calculateFlightHours(fuelPercent, burnRatePerHour) {
  // Guard clause: handle an invalid input instead of returning nonsense
  if (burnRatePerHour <= 0) {
    console.log("Burn rate must be greater than zero.");
    return 0;
  }
  return fuelPercent / burnRatePerHour;
}

// An arrow function equivalent, used for a smaller helper
const formatHours = (hours) => `${hours.toFixed(1)} hours`;

const flightHours = calculateFlightHours(fuelLevel, 6);
console.log("Estimated flight time:", formatHours(flightHours));

// --- LOOPS ---
// for loop: run the checklist exactly crew.length times
console.log("--- Pre-Launch Crew Check ---");
for (let i = 0; i < crew.length; i++) {
  console.log(`Crew member ${i + 1}: ${crew[i]} – status OK`);
}

// while loop: keep counting down until liftoff
let countdown = 5;
console.log("--- Countdown ---");
while (countdown > 0) {

```

```

    console.log(countdown);
    countdown = countdown - 1;
  }
  console.log("Liftoff!");

```

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Lumexa Mission Console</title>
</head>
<body>
  <h1>Open your browser console to see Mission Control output.</h1>
  <script src="mission-console.js"></script>
</body>
</html>

```

Code Walkthrough

- **Variables section:** `shipName` and `crew` are declared with `const` because they never change during the program. `fuelLevel` and `isLaunchReady` use `let` because their values are updated later — trying to reassign a `const` would throw a `TypeError`.
- **Operators section:** `fuelLevel - 4` is arithmetic subtraction. `fuelLevel > 80` is a comparison that evaluates to a boolean (`true` or `false`), which is then stored in `isLaunchReady`.
- **calculateFlightHours:** a standard function declaration with two parameters. The `if` statement is a **guard clause** — it exits early with a safe fallback rather than dividing by zero or producing `Infinity`. The final line uses `return` to send a value back to the caller.
- **formatHours:** an arrow function written on one line since it has a single expression. It uses a **template literal** (backticks with `${}`) to build a string that embeds a variable directly — cleaner than string concatenation with `+`.
- **for loop:** `i` starts at `0`, the loop keeps running while `i < crew.length`, and `i++` increments it after each pass. `crew[i]` accesses the array by its numeric index (arrays are zero-indexed).
- **while loop:** repeats as long as `countdown > 0`, manually decreasing `countdown` inside the loop body. Forgetting that decrement would create an **infinite loop** — a common and important mistake to understand.

Student Mission

Extend `mission-console.js` so that:

1. You add a new `const` array called `systemsCheck` containing at least 4 strings (e.g., `"Oxygen"`, `"Navigation"`, `"Communications"`, `"Engines"`).
2. You write a `for` loop that logs `"<system> check: PASS"` for every item in `systemsCheck`.
3. You write a function `isFuelSafe(fuelPercent)` that returns `true` if fuel is above 20% and `false` otherwise, and call it, logging the result.

Challenge

Write a function `countdownToLaunch(startNumber)` that uses a `while` loop internally (not `console.log` scattered outside it) to return an **array** of the countdown numbers in order, ending with the string `"Liftoff!"` as the last array element. For example, `countdownToLaunch(3)` should return `[3, 2, 1, "Liftoff!"]`.

Experiment/Extension

Try changing your `for` loop's increment from `i++` to `i += 2` and predict what will print before running it. Then try setting the loop condition to something that's never true (e.g., `i < 0`) and observe that the loop body never runs at all — this teaches you that loops are conditional, not automatic.

Common Mistakes

- Using `=` (assignment) instead of `===` (comparison) inside an `if` condition, which accidentally reassigns a variable instead of comparing it.
- Forgetting to update the loop variable in a `while` loop, causing an infinite loop that freezes the browser tab.
- Trying to reassign a `const` variable and being confused by the `TypeError: Assignment to constant variable` error.
- Off-by-one errors in `for` loops, such as using `i <= crew.length` which causes `crew[i]` to be `undefined` on the last pass.
- Forgetting that array indexes start at `0`, not `1`.

Debugging Tips

- Open DevTools with `F12` (or right-click → Inspect) and click the **Console** tab to see your `console.log()` output and any red error messages.
- If the page freezes, you likely wrote an infinite loop — close the tab (or use the browser's "page unresponsive" stop option) and check your loop's condition and increment.
- Click on an error message in red text in the Console — it names the exact file and line number where the problem occurred.
- Try typing expressions directly into the Console (e.g., `2 + 2` or `crew.length`) to experiment without editing your file.

Check Your Understanding

1. What is the difference between `let` and `const`?
2. Why does `crew[0]` refer to "Ada" and not "Grace"?
3. What happens if a `while` loop's condition is never false?
4. What does a function's `return` statement do?

5. Name two primitive data types besides `string` and `number`.

Mini Quiz + Answer Key

Q1. Which keyword should you use by default for a variable that will not be reassigned? A) `var` B) `let` C) `const` D) `function`

Q2. What does `crew.length` return for `["Ada", "Grace"]`? A) `0` B) `1` C) `2` D) `undefined`

Q3. What will `typeof true` evaluate to? A) `"string"` B) `"boolean"` C) `"number"` D) `"object"`

Q4. Which loop type is best when you don't know in advance how many repetitions you need? A) `for` B) `while` C) neither D) both work identically in every case

Answer Key: 1-C, 2-C, 3-B, 4-B

Lesson Recap

You learned how JavaScript stores information in variables (`let` / `const`), the core data types it uses, how to package logic into functions with parameters and return values, and how to automate repetition with `for` and `while` loops — all while using `console.log()` and DevTools to see what your code is actually doing.

Homework

Write a new script, `crew-roster.js`, that stores an array of 5 crew names, a function `introduceCrewMember(name, role)` that returns a formatted greeting string, and a `for` loop that calls that function for each crew member with a role of your choosing, logging each greeting to the console.

Portfolio Project Connection

The variables, functions, and loops you practiced here are the foundation for **all three** course projects. You'll use `const` / `let` to track to-do items (Project 04), loop over forecast days from the weather API (Project 05), and use functions to control the memory-matching game logic (Project 06).